Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Simulate smoothing and sharpening operation on images using spatial filters.**	
Experiment No: 9	**Date:27-08-2026**	**Enrollment No:92400133121**

AIM: Simulate smoothing and sharpening operation on images using spatial filters.

Theory:

Smoothing and sharpening are common image enhancement techniques used to modify the spatial characteristics of an image.

Smoothing:

- Smoothing, also known as blurring, is used to reduce noise and remove fine details in an image.
- It works by applying a low-pass filter that averages the pixel values within a neighbourhood, thus producing a blurred effect.

Sharpening:

- Sharpening is used to enhance the edges and fine details in an image.
- It works by applying a high-pass filter that amplifies the differences between neighbouring pixel values, thus increasing the contrast and emphasizing edges**.**

Flowchart:

- The flowchart for the program will consist of the following steps:
- Load the input image.
- Apply the desired smoothing or sharpening filter to the image.
- Display the original image and the filtered image side by side.
- Save the filtered image (optional).

CODE:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("images/noisy_image.jpg")
plt.figure(figsize=(4,4))
plt.imshow(image)
plt.title("Input image")
plt.axis("off")
plt.show()
```

Marwadi University Faculty of Engineering &Technology Department of Information and Communication Technology		
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Simulate smoothing and sharpening operation on images using spatial filters.**	
Experiment No: 9	**Date:27-08-2026**	**Enrollment No:92400133121**



MEAN FILTER

```
mean_3x3 = cv2.blur(image,(3,3))  #3x3, 5*5, etc  for 3x3 filter-->every coefficient is 1/9

mean_5x5 = cv2.blur(image,(5,5))

plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(image)
plt.title("Original Image")
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(cv2.cvtColor(mean_3x3,cv2.COLOR_BGR2RGB))
plt.title("Mean Filter (3x3) Image")
plt.axis('off')

plt.subplot(1,3,3)
plt.imshow(cv2.cvtColor(mean_5x5,cv2.COLOR_BGR2RGB))
plt.title("Mean Filter (5x5) Image")
plt.axis('off')
```

plt.show()



Conclusion:

- Each pixel value is replaced with the average of its neighbours.
- As kernel increases the image becomes more blurred.

WEIGHTED MEAN FILTER

```
weighted_kernel = np.array([
   [1,1,2],
   [1,3,1],
   [1,1,7]
],dtype=np.float32)

weighted_kernel = weighted_kernel / 18
weighted_mean = cv2.filter2D(image,-1,weighted_kernel)

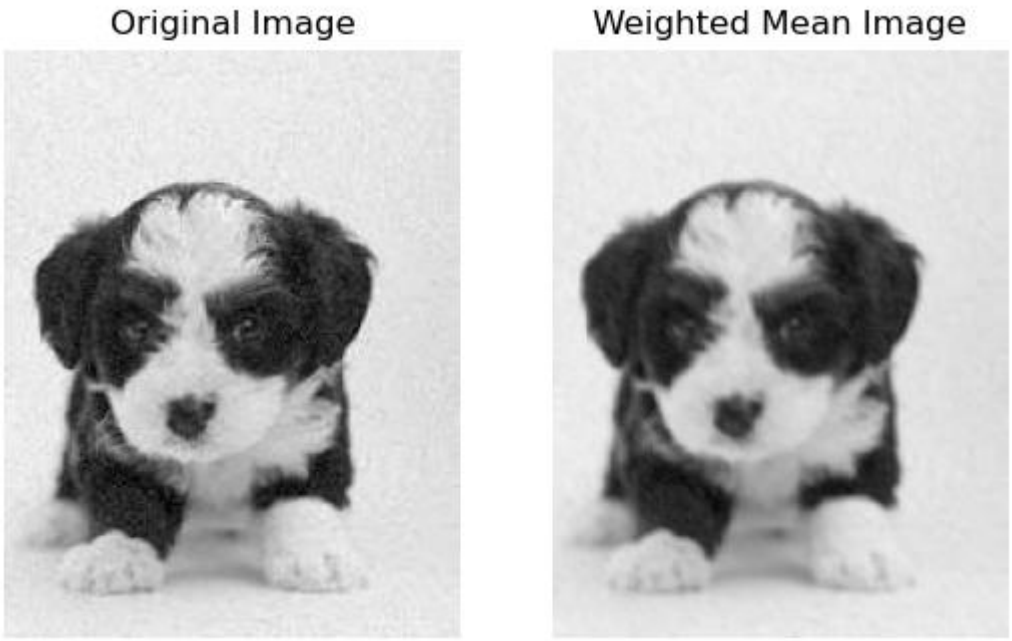
plt.figure(figsize=(10,6))

plt.subplot(1,3,1)
plt.imshow(image)
plt.title("Original Image")
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(cv2.cvtColor(weighted_mean,cv2.COLOR_BGR2RGB))
```

```
plt.title("Weighted Mean Image")
plt.axis('off')

plt.show()
```



Conclusion:

This filter allows specific pixel positions within the neighbourhood to contribute more to final pixel value.
This can be where directional smoothing is required.

GUASSIAN MEAN FILTER

```
#method-1

guassian_kernel = np.array([
    [1,2,1],
    [2,4,2],
    [1,2,1]
],dtype=np.float32)

guassian_kernel = guassian_kernel / 16
```

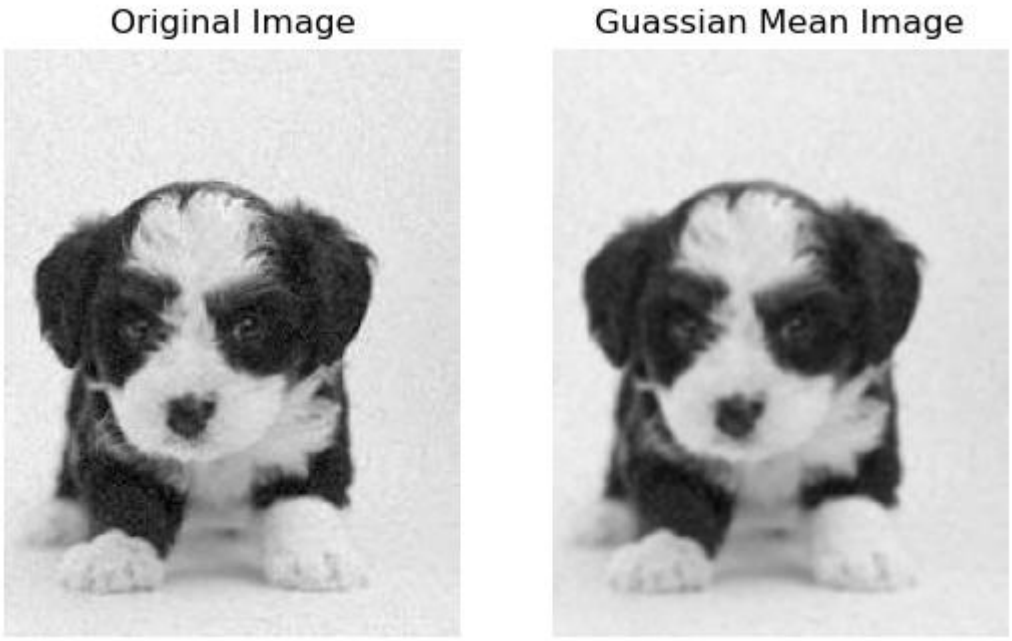
```
guassian_mean = cv2.filter2D(image,-1,guassian_kernel)

plt.figure(figsize=(10,6))

plt.subplot(1,3,1)
plt.imshow(image)
plt.title("Original Image")
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(cv2.cvtColor(guassian_mean,cv2.COLOR_BGR2RGB))
plt.title("Guassian Mean Image")
plt.axis('off')

plt.show()
```



ALTERNATIVE METHOD

```
#method-2
g0 = cv2.GaussianBlur(image,(5,5),sigmaX=0)

g1 = cv2.GaussianBlur(image,(5,5),sigmaX=1)
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Simulate smoothing and sharpening operation on images using spatial filters.**	
Experiment No: 9	**Date:27-08-2026**	**Enrollment No:92400133121**

```
g2 = cv2.GaussianBlur(image,(5,5),sigmaX=2)

plt.figure(figsize=(10,6))

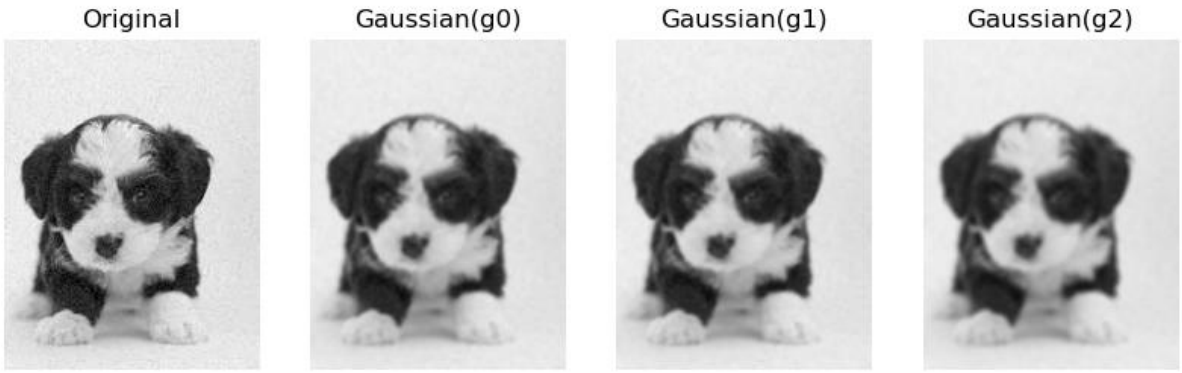
plt.subplot(1,4,1)
plt.imshow(image)
plt.title("Original")
plt.axis('off')

plt.subplot(1,4,2)
plt.imshow(cv2.cvtColor(g0,cv2.COLOR_BGR2RGB))
plt.title("Gaussian(g0)")
plt.axis('off')

plt.subplot(1,4,3)
plt.imshow(cv2.cvtColor(g1,cv2.COLOR_BGR2RGB))
plt.title("Gaussian(g1)")
plt.axis('off')

plt.subplot(1,4,4)
plt.imshow(cv2.cvtColor(g2,cv2.COLOR_BGR2RGB))
plt.title("Gaussian(g2)")
plt.axis('off')

plt.show()
```



Conclusion

In this case the weights decrease as you move away from the centre pixel.

Increasing sigmaX gives more natural blue and smoother image

LAPLACIAN EDGE DETECTION

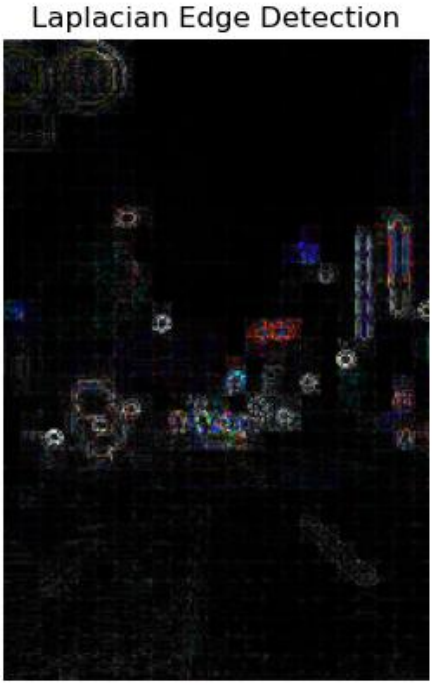
```
image1 = cv2.imread("images/image1.jpg")
plt.figure(figsize=(4,4))
plt.imshow(image1)
plt.title("Input image")
plt.axis("off")
plt.show()
```




```
laplacian = cv2.Laplacian(image1,cv2.CV_32F)
#applying absolute to get always +ve value
laplacian_abs = cv2.convertScaleAbs(laplacian)

plt.imshow(laplacian_abs,cmap='gray')
plt.title('Laplacian Edge Detection')
plt.axis('off')
plt.show()
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Simulate smoothing and sharpening operation on images using spatial filters.**	
Experiment No: 9	**Date:27-08-2026**	**Enrollment No:92400133121**



Conclusion:

This filter turns the background black and isolates sharp intensity transitions, transforming them into bright white outlines.
It is best used for edge detection, structural feature extraction.